PRUEBA TÉCNICA SPRING BOOT ANGULAR

ECONOCOM

2025

RESTRICTED CIRCULATION

econocom
LIVE TECH

econocom
LIVE TECH

Tabla de contenidos

Contenido

1 Introducción

1.1 Requisitos del Backend (Spring Boot):
1.2 Requisitos del Frontend (Angular):
1.3 Requisitos de Conexión (Integración):
1.4 Criterios de Evaluación:
1.5 Tiempo Estimado:

2 Ampliación - Funcionalidad SSO al Login

2.1 Requisitos Adicionales del Backend (Spring Boot):
2.2 Requisitos Adicionales del Frontend (Angular):
2.3 Consideraciones Adicionales:
2.4 Criterios de Evaluación Adicionales:
2.5 Tiempo Estimado (con SSO):

3 Entrega y recursos

1 Introducción

Esta prueba tiene como objetivo evaluar las habilidades técnicas de un candidato en el desarrollo full-stack utilizando Spring Boot para el backend y Angular para el frontend. La prueba se centrará en la capacidad de diseñar, implementar y conectar ambas partes para generar una interfaz de inicio de sesión funcional y visualmente similar al diseño proporcionado.

1.1 Requisitos del Backend (Spring Boot):

1. **Configuración del Proyecto:**
   - Crear un proyecto Spring Boot utilizando Spring Initializr o la herramienta de su preferencia.
   - Utiliza la versión de Java 1.8.
   - Incluir las dependencias necesarias para la creación de una API REST (Spring Web).
2. **Endpoint de Autenticación:**
   - Desarrollar un endpoint REST (/api/auth/login) que reciba las credenciales del usuario (correo electrónico y contraseña) mediante una petición POST.
   - Implementar una lógica de autenticación JWT (no se requiere conexión a una base de datos real para esta prueba).
   - Implementar lógica de manejo de la caducidad del token, así como el refresco y validación de este.
3. **Estructura del Código:**
   - Organizar el código de manera clara y siguiendo las buenas prácticas de desarrollo en Spring Boot (uso de controladores, servicios, repositorios etc.).
   - Incluir comentarios relevantes para facilitar la comprensión del código.

1.2 Requisitos del Frontend (Angular):

1. **Configuración del Proyecto:**
   - Crear un proyecto Angular utilizando Angular CLI (16.2.16).
   - El proyecto deberá utilizar Material (16.2.14).
2. **Componente de Login:**
   - Crear un componente Angular responsable de renderizar la interfaz de inicio de sesión, replicando la estructura visual de la imagen proporcionada. Esto incluye:
     - Campos de entrada para "Correo electrónico" y "Contraseña".
     - Un botón "ENTRAR".
     - Integración de la imagen especificada en el diseño.
     - Aplicación de estilos CSS para lograr una apariencia similar a la imagen (se recomienda utilizar la librería proporcionada).

3. **Servicio de Autenticación:**

- Crear un servicio Angular para comunicarse con el endpoint de autenticación del backend.
- Este servicio deberá enviar una petición POST al endpoint /api/auth/login con los datos ingresados por el usuario en el formulario.
- Manejar la respuesta del backend:
  - En caso de éxito, mostrar un mensaje (puede ser un console.log o una alerta simple).
  - En caso de error, mostrar un mensaje de error al usuario en la interfaz.

4. **Validación del Formulario:**
   - Implementar validaciones básicas en el formulario de inicio de sesión (campos requeridos, formato de correo electrónico).
5. **Estructura del Código:**
   - Organizar el código de manera modular y siguiendo las buenas prácticas de desarrollo en Angular (uso de componentes, servicios, módulos, etc.).
   - Incluir comentarios relevantes para facilitar la comprensión del código.
   - Utilizar la librería CSS proporcionada.

1.3 Requisitos de Conexión (Integración):

1. **Comunicación HTTP:**
   - El frontend Angular debe utilizar el módulo HttpClient para realizar la petición POST al backend Spring Boot.
2. **Manejo de CORS (si es necesario):**
   - Configurar el backend Spring Boot para permitir las peticiones desde el dominio donde se ejecute el frontend Angular (si ambos se ejecutan en puertos diferentes localmente).

1.4 Criterios de Evaluación:

- **Funcionalidad:** Que la interfaz de inicio de sesión permita introducir credenciales y comunicarse correctamente con el backend.
- **Implementación del Backend:** Correcta implementación del endpoint de autenticación y manejo de las peticiones. Se valorará también el uso de los ORM MyBatis o JPA y la compatibilidad con Java 1.8.
- **Implementación del Frontend:** Correcta creación del componente de login, el servicio de autenticación y la comunicación con el backend.
- **Calidad del Código:** Claridad, organización, legibilidad y seguimiento de buenas prácticas en ambos proyectos. Se valorará la utilización de BEM y atomic design para la estructura de scss así como el uso de la librería proporcionada.
- **Diseño Visual (Similitud):** Que la interfaz de login del frontend se asemeje razonablemente al diseño proporcionada.
- **Manejo de Errores:** Correcto manejo de las respuestas de error del backend y presentación de mensajes informativos al usuario.

1.5 Tiempo Estimado:

Se estima que esta prueba técnica puede llevar entre 4 y 8 horas de dedicación.

2 Ampliación - Funcionalidad SSO al Login

Además de los requisitos iniciales, se deberá implementar la funcionalidad de Single Sign-On (SSO) en el proceso de inicio de sesión. Para simplificar la prueba, se simulará un proveedor de SSO.

2.1 Requisitos Adicionales del Backend (Spring Boot):

1. **Endpoint de Redirección SSO:**
   - Crear un nuevo endpoint (/api/auth/sso) que simule el inicio del flujo de autenticación con un proveedor de SSO.
   - Este endpoint deberá generar una URL de redirección simulada al proveedor de SSO. Esta URL podría contener parámetros como un client_id simulado y una redirect_uri de vuelta a la aplicación.
   - La respuesta de este endpoint será una redirección HTTP (código 302) a la URL del proveedor de SSO simulado.
2. **Endpoint de Callback SSO:**
   - Crear un endpoint de callback (/api/auth/sso/callback) que simule recibir la respuesta del proveedor de SSO después de una autenticación exitosa (o fallida).
   - Este endpoint recibirá parámetros en la URL (e.g., un code de autorización simulado).
   - Implementar la lógica para:
     - Simular la validación del código de autorización con el proveedor de SSO simulado (no se requiere una implementación real).
     - En caso de validación exitosa simulada, generar una respuesta JSON indicando éxito (similar al login tradicional, podría incluir un token simulado o un mensaje).
     - En caso de fallo simulado, devolver una respuesta JSON con un código de error y un mensaje descriptivo.

2.2 Requisitos Adicionales del Frontend (Angular):

1. **Botón de Inicio de Sesión SSO:**
   - Añadir un nuevo botón a la interfaz de login con una etiqueta como "Iniciar sesión con SSO".
   - Al hacer clic en este botón, la aplicación Angular deberá realizar una petición GET al endpoint /api/auth/sso del backend.
   - Manejar la respuesta de redirección del backend, redirigiendo al usuario a la URL proporcionada por el backend (el proveedor de SSO simulado).
2. **Manejo del Callback SSO:**
   - Implementar una ruta en la aplicación Angular que corresponda a la redirect_uri configurada en el backend (e.g., /sso/callback).
   - Crear un componente Angular asociado a esta ruta que se active cuando el usuario es redirigido de vuelta por el proveedor de SSO simulado.

- Este componente deberá leer los parámetros de la URL (e.g., el code simulado) y enviar esta información al endpoint de callback del backend (/api/auth/sso/callback).
- Manejar la respuesta del backend (éxito o error) y mostrar el resultado al usuario (similar al manejo de la respuesta del login tradicional).

2.3 Consideraciones Adicionales:

- **Proveedor de SSO Simulado:** No es necesario interactuar con un proveedor de SSO real. Se puede simular todo el flujo en el backend. Por ejemplo, el endpoint /api/auth/sso podría simplemente generar una URL con un código de autorización fijo y redirigir a /api/auth/sso/callback con ese código.
- **Seguridad (Simulada):** El objetivo es demostrar la comprensión del flujo general.

2.4 Criterios de Evaluación Adicionales:

- **Implementación del Flujo SSO en el Backend:** Correcta implementación de los endpoints de redirección y callback, y la simulación de la interacción con el proveedor de SSO.
- **Implementación del Flujo SSO en el Frontend:** Correcta implementación del botón de inicio de sesión SSO, la redirección al proveedor simulado y el manejo del callback.
- **Comunicación entre Frontend y Backend para SSO:** Correcta transmisión de información entre ambas partes durante el flujo de SSO.
- **Comprensión del Flujo SSO:** Demostrar una comprensión básica del proceso de Single Sign-On.

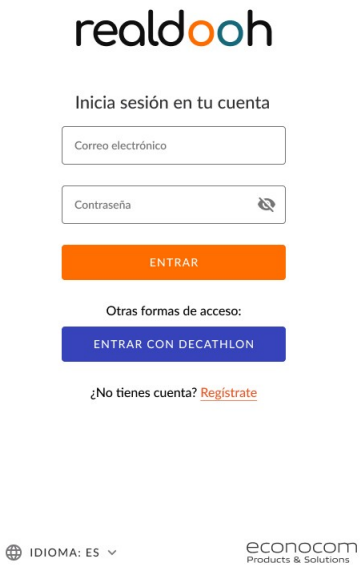
2.5 Tiempo Estimado (con SSO):

Se estima que la adición de la funcionalidad SSO podría extender el tiempo de dedicación de la prueba en 2 a 4 horas adicionales, llevando el tiempo total estimado a entre 6 y 12 horas.

Esta ampliación de la prueba técnica permite evaluar la capacidad del candidato para integrar la funcionalidad de Single Sign-On en una aplicación web utilizando Spring Boot y Angular, comprendiendo el flujo básico de este tipo de autenticación.

3 Entrega y recursos

El candidato deberá entregar un repositorio Git que contenga tanto el código del backend Spring Boot como el del frontend Angular. Se valorará la inclusión de instrucciones claras para ejecutar ambas aplicaciones.

Carpeta de assets: Se adjunta como technical-challenge.zip

Enlace al diseño: Figma (necesario solicitar acceso)